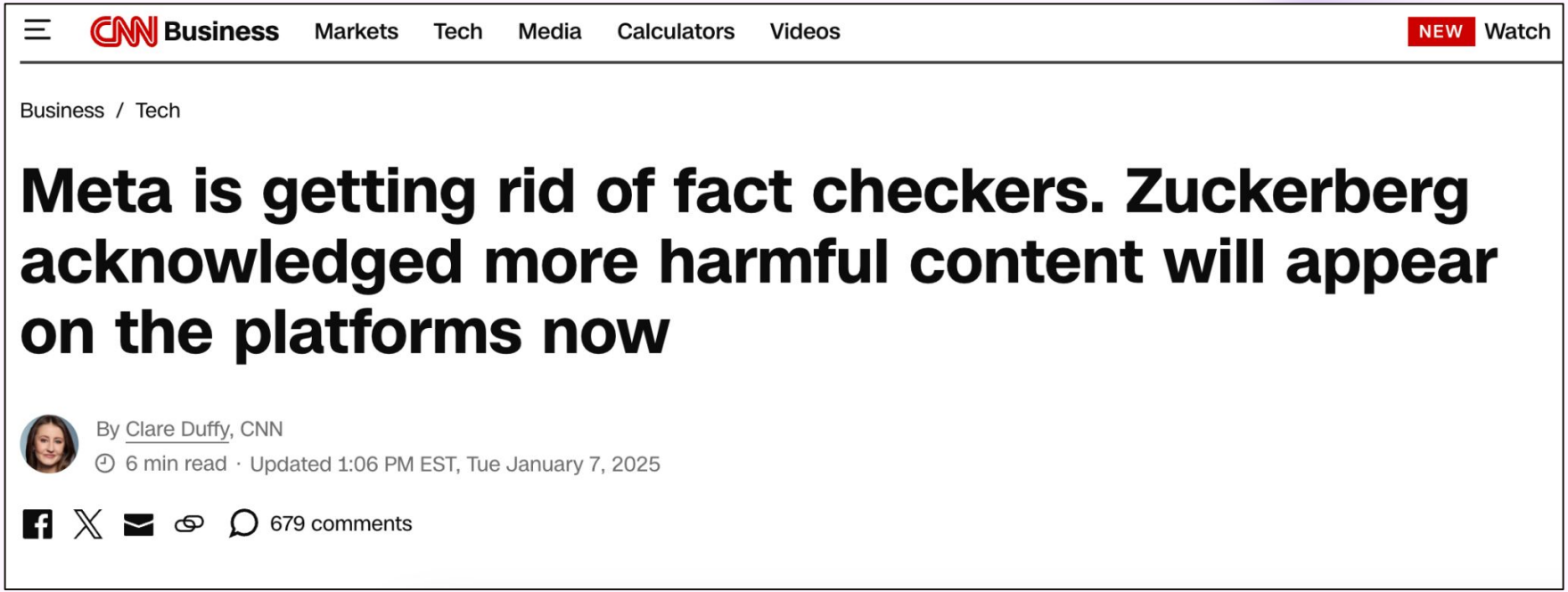
Nolan, Rhea, Olivia, Natalie & Julien

Bullshit Intervention

What is bullshit?

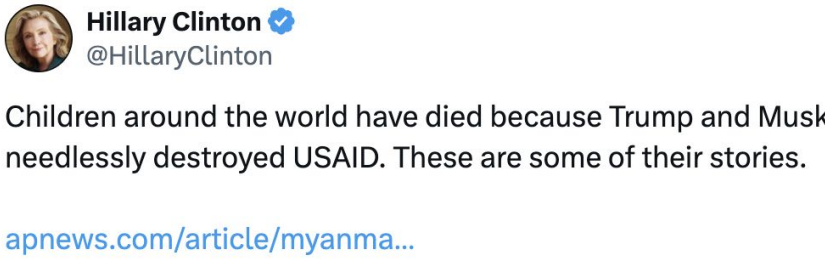
Bullshit is communication that mimics sincere, truth-based assertion while being indifferent to whether what is said is true or false, aiming instead at producing a desired effect [often chaos] on an audience. Unlike lying, bullshit does not (intentionally) distort the truth—it disregards it.



How can we avoid falling for bullshit tweets? How can we avoid contributing to bullshit online?

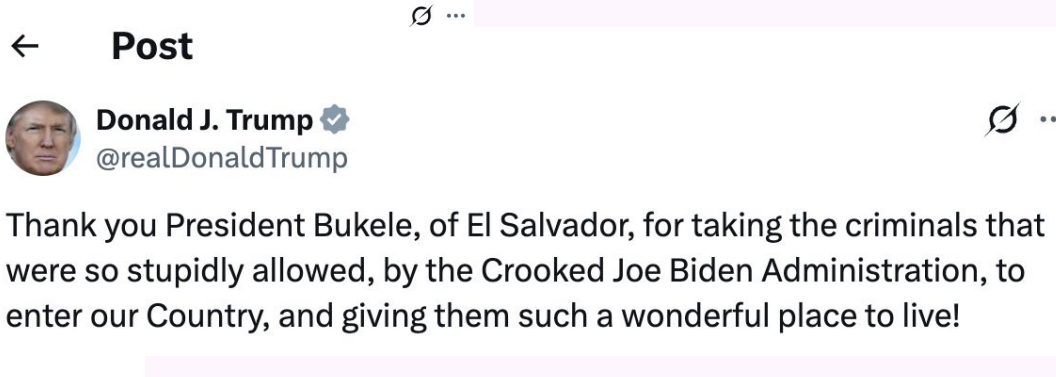








What is the solution....?



How could one detect bullshit?

1. **Given a tweet you are evaluating (let's call this the "draft")**
    a. Have an awareness of
        i. The facts of the topic
        ii. What the speaker has asserted is true in the past
    b. Evaluate the **consistency** of the "draft" to these two contexts
        i. If the speaker's truth and what is "factually" true match
            1. Draft matches contexts = **not bullshit**
            2. Draft contradicts both contexts = **bullshit**
        ii. If speaker truth contradicts "factual" truth
            1. Draft matches speaker truth: **not bullshit** (even if false)
            2. Draft contradicts speaker truth
                a. but follows "factual" truth: wildcard? **not bullshit**
                b. But contradicts factual truth: **bullshit**

Consistency is essentially how we are evaluating "indifference" to the truth

- Is the "draft" random considering what the user believes?

Future Additions/ Ideas

1. **Better context metrics**
    a. Is the user a serial bullshitter/shitposter? Is there alignment across other forms of the users' digital presence (LinkedIn, interviews, papers).

2. **Bullshit score calculator**
    a. Numerical scale displaying a "bullshit value", visible on every post

3. **Algorithmic implications**
    a. Push posts with lower bullshit scores, higher bullshit scores won't reach audiences

4. **Other forms of media?**
    a. Video, audio, or image bullshit detection technology?

Implementation: **Prepost AI**

1. **Checks Past Posts**
   a. Reviews the user previous tweets
   b. Prioritizes consistency + avoid self-contradictions
   c. Produces similarity scores

```python
# ------------------------------
# 1. MODEL INITIALIZATION
# ------------------------------

embedder = SentenceTransformer("sentence-transformers/all-mpnet-base-v2")

nli_name = "roberta-large-mnli"
nli_tokenizer = AutoTokenizer.from_pretrained(nli_name)
nli_model = AutoModelForSequenceClassification.from_pretrained(nli_name)

device = "cuda" if torch.cuda.is_available() else "cpu"
nli_model.to(device)

# LLM / agent model (via LangChain + LangGraph)
access_token = get_access_token()
chat = ChatOpenAI(
    model="google/gemma-3-27b-it",
    openai_api_key=access_token,
    openai_api_base="https://inference-api.alcf.anl.gov/resource_server/sophia/vllm/v1",
)
```

```python
# 3. EXTERNAL CONTEXT TOOLS
# ------------------------------

def fetch_wikipedia(query: str) -> str:
    url = f"https://en.wikipedia.org/api/rest_v1/page/summary/{query.replace(' ', '_')}"
    try:
        r = requests.get(url, timeout=5)
        if r.status_code == 200:
            return r.json().get("extract", "")
    except Exception:
        pass
    return ""

def fetch_bbc(query: str, max_results: int = 2) -> List[str]:
    url = f"https://www.bbc.co.uk/search?q={query}"
    try:
        r = requests.get(url, timeout=5)
        if r.status_code != 200:
            return []
        soup = BeautifulSoup(r.text, "html.parser")
        results = []
        for a in soup.select("article h1 a")[:max_results]:
            href = a.get("href", "")
            headline = a.get_text(strip=True)
            try:
                art = requests.get(href, timeout=5)
                art_soup = BeautifulSoup(art.text, "html.parser")
                paras = " ".join(p.get_text() for p in art_soup.find_all("p")[:3])
                results.append(f"{headline}: {paras}")
            except Exception:
                continue
        return results
    except Exception:
        return []
```

2. **Fact Retrieval**
   a. Identifies key topics in the draft → pulls info from trusted sources

Prepost AI

3. **Contradiction Detection (NLI)**
   a. Natural language inference to compare the draft with verified facts + user's history
      i. Flags any contradictions/ unsupported claims

```python
# ----------------------------
# 2. USER MEMORY / FAISS
# ----------------------------

user_data = {}
user_indices = {}

def get_or_create_index(user_id: str):
    if user_id not in user_indices:
        user_indices[user_id] = faiss.IndexFlatIP(768)
    return user_indices[user_id]

def add_user_texts(user_id: str, texts: List[str]):
    if user_id not in user_data:
        user_data[user_id] = {"texts": []}
    idx = get_or_create_index(user_id)
    embeddings = embedder.encode(texts, normalize_embeddings=True)
    idx.add(np.array(embeddings, dtype="float32"))
    user_data[user_id]["texts"].extend(texts)

def retrieve_similar(user_id: str, draft: str, top_k: int = 5) -> List[str]:
    if user_id not in user_data:
        return []
    idx = get_or_create_index(user_id)
    draft_emb = embedder.encode([draft], normalize_embeddings=True)
    D, I = idx.search(np.array(draft_emb, dtype="float32"), top_k)
    texts = user_data[user_id]["texts"]
    return [texts[i] for i in I[0] if i < len(texts)]
```

4. **Risk Evaluation / Classification**
   a. Large language model evaluates all info + risk level
      i. Low / Medium / High
   b. Explains inaccuracy/risky
   c. Classifies "bullshit", "not bullshit", "contextually ambiguous"

```python
def display_results(result: Dict[str, Any], system_name: str):
    print("\n" + "="*70)
    print("FINAL RESULTS")
    print("="*70)

    llm_resp = result.get("llm_response", {})

    classification = llm_resp.get("classification", "unknown").upper()
    confidence = llm_resp.get("confidence", "unknown").upper()
    reasoning = llm_resp.get("reasoning", "No reasoning provided")
    suggested = llm_resp.get("suggested_revision", "")

    # Color code classification
    if "bullshit" in classification.lower():
        class_display = f"🚨 {classification}"
    elif "not_bullshit" in classification.lower():
        class_display = f"✅ {classification}"
    else:
        class_display = f"⚠️  {classification}"

    print(f"\n📊 CLASSIFICATION: {class_display}")
    print(f"📈 CONFIDENCE: {confidence}")
    print(f"\n💭 REASONING:")
    print(f"   {reasoning}")

    if suggested:
        print(f"\n✏️  SUGGESTED REVISION:")
        print(f"   {suggested}")
```

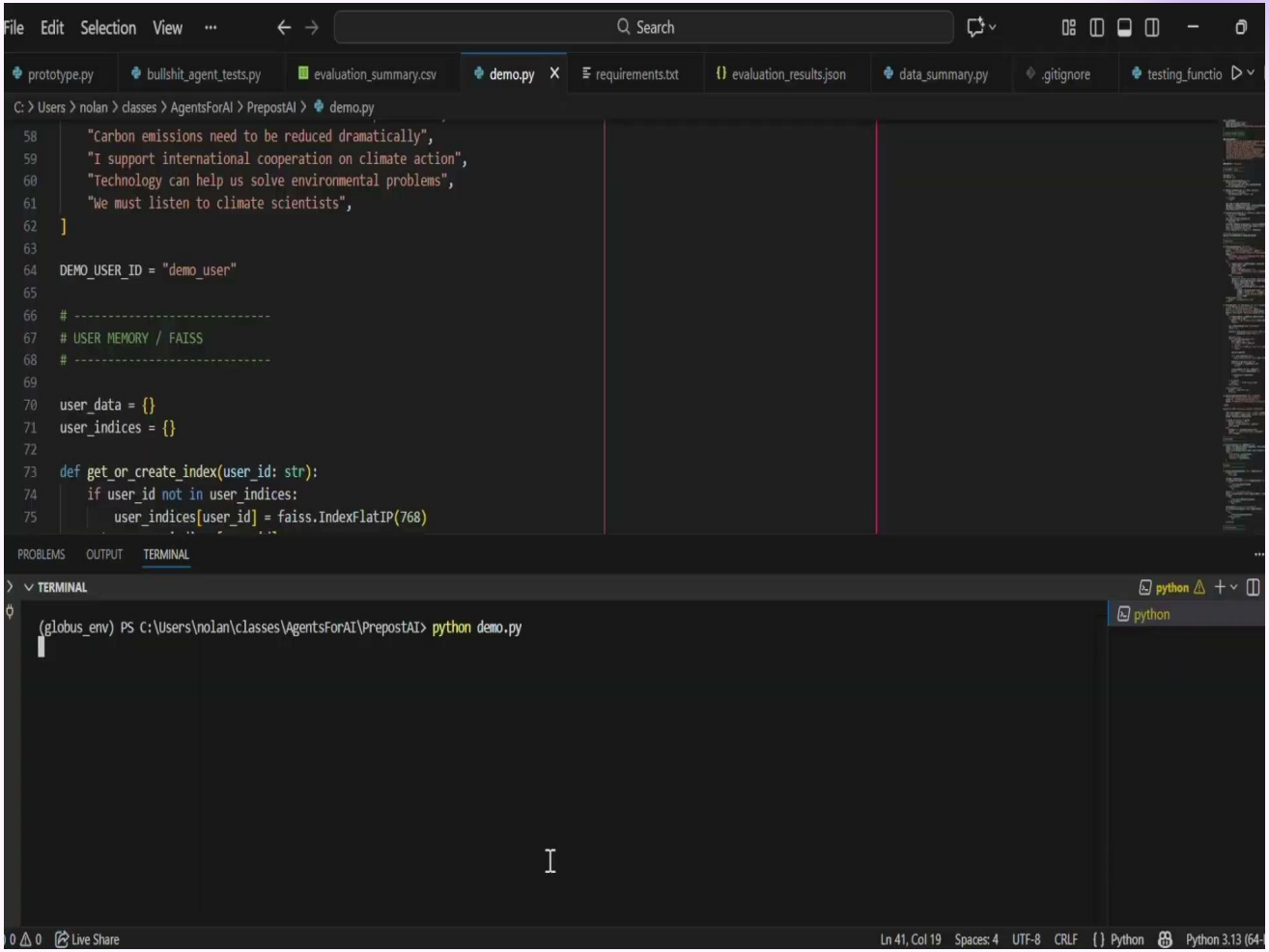
Prepost AI

5. Safer Rewrite

- a. If needed → the system generates a revised version of the post
  - i. Maximize clarity, accuracy, and minimize misinformation

```
611         if suggested:
612             print(f"\n✏️  SUGGESTED REVISION:")
613             print(f"   {suggested}")
614 
615         # Show user context that was retrieved
616         user_contexts = result.get("user_contexts", [])
617         if user_contexts:
618             print(f"\n👤 USER'S PREVIOUS POSTS REFERENCED ({len(user_contexts)}):")
619             for i, ctx in enumerate(user_contexts, 1):
620                 print(f"   {i}. \"{ctx}\"")
621 
622         # Show external sources
623         external_contexts = result.get("external_contexts", [])
624         if external_contexts:
625             print(f"\n🌐 EXTERNAL SOURCES CONSULTED ({len(external_contexts)}):")
626             for i, ctx in enumerate(external_contexts[:5], 1):
627                 # Extract just the source name and first part
628                 if ctx.startswith("Wikipedia("):
629                     parts = ctx.split("): ", 1)
630                     if len(parts) == 2:
631                         source = parts[0] + ")"
632                         content = parts[1][:200]
633                         print(f"   {i}. {source}")
634                         print(f"      \"{content}...\"")
635                     else:
636                         print(f"   {i}. {ctx[:250]}...")
637                 elif ctx.startswith("BBC News:"):
638                     print(f"   {i}. {ctx}")
639                 else:
640                     print(f"   {i}. {ctx[:250]}...")
```

Video Demo


File Edit Selection View ···
Search
prototype.py
bullshit_agent_tests.py
evaluation_summary.csv
demo.py
requirements.txt
evaluation_results.json
data_summary.py
.gitignore
testing_functio
C: > Users > nolan > classes > AgentsForAI > PrepostAI > demo.py
"Carbon emissions need to be reduced dramatically",
"I support international cooperation on climate action",
"Technology can help us solve environmental problems",
"We must listen to climate scientists",
]
DEMO_USER_ID = "demo_user"

USER MEMORY / FAISS
----
user_data = {}
user_indices = {}
def get_or_create_index(user_id: str):
if user_id not in user_indices:
user_indices[user_id] = faiss.IndexFlatIP(768)
PROBLEMS OUTPUT TERMINAL
TERMINAL
python
(globus_env) PS C:\Users\nolan\classes\AgentsForAI\PrepostAI> python demo.py
Live Share
Ln 41, Col 19 Spaces: 4 UTF-8 CRLF { } Python Python 3.13 (64-

Questions?